

# Raymarine

Rétro-ingénierie du réseau et des protocoles MFD

Axiom · LightHouse · RayConnect

Compilation des notes techniques du dépôt

14/09/2026

# Sommaire

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Rétro-ingénierie de <b>ray5800</b> — Multicast UDP 5800 .....           | 3  |
| 1. Cadre réseau (●) .....                                               | 3  |
| 2. Les trois types de message (●) .....                                 | 3  |
| 3. Trame d'annonce (types 1 & 2) (●) .....                              | 3  |
| 4. Trame de type 0 (annonce de service / canal) (●) .....               | 4  |
| 5. Adressage : unicast RayNet + groupes multicast (●) .....             | 6  |
| 6. Comportement temporel (●) .....                                      | 6  |
| 7. Questions ouvertes .....                                             | 6  |
| Protocole RayDB — TCP 23333 (Raymarine) .....                           | 7  |
| 1. Vue d'ensemble .....                                                 | 7  |
| 2. Cycle de vie d'une connexion .....                                   | 7  |
| 3. Format d'une trame .....                                             | 8  |
| 4. Opcodes ( op ) .....                                                 | 8  |
| 5. Bloc valeur .....                                                    | 9  |
| 7. Espace de noms des chemins .....                                     | 10 |
| 8. Correspondance RayDB → NMEA 0183 .....                               | 10 |
| 9. Notes d'implémentation .....                                         | 11 |
| Protocole RRCE — télécommande Raymarine (TCP 50000) .....               | 12 |
| Vue d'ensemble .....                                                    | 12 |
| Format de trame .....                                                   | 12 |
| Structure d'un geste .....                                              | 15 |
| Séquences types .....                                                   | 15 |
| Outillage du projet .....                                               | 15 |
| Points de sécurité .....                                                | 15 |
| Analyse du SSH du MFD Raymarine — synthèse exhaustive .....             | 16 |
| Ouvrir une session SFTP .....                                           | 16 |
| Protocole de messages Raymarine — TCP 8182 .....                        | 17 |
| 1. Ce que résout ce protocole .....                                     | 17 |
| 2. Cadre réseau (●) .....                                               | 17 |
| 3. Format de trame (●) .....                                            | 17 |
| 5. La clé enrôlée = la clé de RayConnect (●) .....                      | 18 |
| 6. Côté client : d'où vient la trame (●) .....                          | 18 |
| 7. Sécurité : enrôlement en clair, accepté sans étape visible (●) ..... | 19 |
| 8. Questions — résolues et ouvertes .....                               | 20 |

# Rétro-ingénierie de ray5800 — Multicast UDP 5800

**ray5800** est le protocole d'annonce et de découverte des équipements Raymarine sur Ethernet. Ce nom est **le nôtre**, faute de nom officiel connu : *SeaTalk-HS* (SeaTalkHS, SeaTalk High Speed) désigne le **réseau physique** sur lequel il circule, pas le protocole.

**Niveau de confiance** : ● confirmé par les données · ● hypothèse forte · ● non résolu.

## 1. Cadre réseau (●)

| Élément     | Valeur                                                                 |
|-------------|------------------------------------------------------------------------|
| Émetteur    | MFD, port source éphémère                                              |
| Destination | 224.0.0.1 (multicast all-systems), port 5800                           |
| Sens        | <b>push unidirectionnel</b> (aucune réponse observée)                  |
| Cadence     | ~0,72 s par équipement (beacon régulier, min 1 ms en rafale, max ~5 s) |

Chaque MFD relaie sur le Wi-Fi captif les équipements qu'il voit sur le **backbone interne SeaTalk-HS/RayNet** (adressage 198.18.0.0/21, cf. §5). Deux MFD sur le même bus n'ont pas forcément la même vue : seul le **maître des données** annonce le radar.

## 2. Les trois types de message (●)

Chaque payload commence par un `type` sur **u32 little-endian**.

| type | Taille(s)      | Rôle                                      |
|------|----------------|-------------------------------------------|
| 1    | 56 o           | <b>Annonce d'équipement</b> (compacte)    |
| 2    | 70 o           | <b>Annonce d'équipement</b> (étendue)     |
| 0    | 28 / 36 / 40 o | <b>Annonce de service</b> (canal IP:PORT) |

Les types 1 et 2 sont en réalité **un seul format à queue variable** (cf. §3). Le type 0 est lui aussi de l'**annonce** — chaque trame publie un ou deux **endpoints IP:PORT** (cf. §4) ; rien n'y est une mesure, à l'exception d'un u16 qui varie sur les records radar.

## 3. Trame d'annonce (types 1 & 2) (●)

Le **type 1 fait toujours 56 octets**, le **type 2 toujours 70**. Ce ne sont pas deux formats : c'est le *même* enregistrement, avec une **queue préfixée par sa longueur**. @52 (u16 LE) donne le nombre d'octets à partir de @54 :

| type | @52         | queue   | total |
|------|-------------|---------|-------|
| 1    | 0x0002 = 2  | @54..55 | 56 o  |
| 2    | 0x0010 = 16 | @54..69 | 70 o  |

Le type 2 ajoute donc exactement **14 octets** (16 - 2), en @56..69.

| offset | champ      | type    | description                                              |
|--------|------------|---------|----------------------------------------------------------|
| 0      | type       | u32 LE  | = 1 (compacte) ou 2 (étendue)                            |
| 4      | u1         | u32 LE  | <span style="color: red;">●</span> NON RÉSOLU — cf. §3.4 |
| 8      | handle     | 4 o     | identifiant UNIQUE du device (opaque, stable)            |
| 12     | descriptor | u32     | type/modèle du device (voir §3.1)                        |
| 16     | ip         | 4 o LE  | IPv4 du device, little-endian                            |
| 20     | name       | ASCIIIZ | nom, terminé par 0x00 (buffer réutilisé, cf. §3.2)       |
| 52     | trail_len  | u16 LE  | nb d'octets à partir de @54 : 2 (type 1) / 16 (type 2)   |

|    |      |      |                                           |
|----|------|------|-------------------------------------------|
| 54 | flag | u8   | 0 ou 1 (puis @55 = 0x00)                  |
| 56 | ext  | 14 o | extension du type 2 uniquement (cf. §3.3) |

### 3.1 handle (@8) vs descriptor (@12) ●

- **handle** : identifiant **unique par device physique**, opaque, **non dérivé de l'IP**, et **stable quel que soit le MFD** qui relaie l'annonce (un même device remonte le même handle depuis deux MFD différents).
- **descriptor** : **type/modèle — partagé par des devices identiques** (deux nœuds instruments de même modèle portent le même descriptor). Décomposition :

```
descriptor = [ @12 code sous-type ][ @13 = 0x00 ][ @14..15 mot de classe ]
              0b 84 (nœud) | 00 00 (radar)
```

En **type 1 seulement**, **u1** suit la même partition : une valeur pour les nœuds instruments, une autre pour les radars. En type 2 la correspondance tombe (cf. §3.4).

### 3.2 Champ name : buffer réutilisé non nettoyé ●

Le nom est un **ASCIIZ** : lire jusqu'au **premier 0x00**, **ignorer la suite**. Le firmware n'écrase le buffer d'annonce **que jusqu'au nul** ; les octets qui suivent sont le **rémanent** d'une écriture précédente dans le même tampon (on observe par exemple un octet résiduel d'un nom plus long derrière un nom plus court). Corollaire : les octets post-nul sont du **contenu résiduel** et **ne doivent pas** être interprétés comme des données.

### 3.3 Extension du type 2 : 14 octets en @56 ●

Les 14 octets (indexés 0..13) portent :

- **[6] (@62) est le sélecteur de variante** : 0x08 = nœud, 0x02 = radar. C'est lui qui indique si [8:14] doit se lire comme un endpoint.
- **En variante radar**, [8:14] porte un **endpoint IP:PORT** sur le bus interne **RayNet 198.18.0.0/21**, au même format qu'en type 0 (§4) : [8:12] l'IP, [12:14] le **port** unicast. C'est l'apport du type 2 pour le radar : celui-ci annonce en @16 l'IP de **son propre point d'accès** (hors RayNet), et l'extension livre son adresse RayNet. Les nœuds annoncent déjà une adresse RayNet en @16 et leur [8:14] ne porte pas d'endpoint.
- Le port publié en [12:14] est **le même** que le port unicast du record de type 0 du même device — recoupement indépendant entre les deux formats, qui confirme que ce champ est un port.
- **Inexpliqués** : [0:4], et chez les nœuds [8:14] (qui ne se lit pas comme un endpoint).

### 3.4 Le champ u1 (@4) et le flag (@54) : non résolus ●

**u1 change avec le type de message** chez les nœuds (une valeur en type 1, une autre en type 2), mais **pas chez les radars** (identique dans les deux). Ce n'est donc ni une longueur, ni un marqueur de classe stable, et il n'est pas lié au modèle : deux devices de même **descriptor** peuvent différer en type 2. Seule corrélation entrevue, sur trop peu de devices pour conclure : chez les nœuds, la valeur de **u1** en type 2 semble suivre celle du **flag**.

Le **flag** (@54, valeurs 0 ou 1) est lui aussi ● non résolu.

## 4. Trame de type 0 (annonce de service / canal) ●

Ce **n'est pas de la télémétrie**. Chaque trame de type 0 publie **un ou deux endpoints IP:PORT** : le device annonce sur quelle adresse et quel port un de ses services écoute ou diffuse. C'est le pendant « canal » des annonces d'équipement des types 1 et 2.

En-tête **fixe de 20 octets**, puis une payload faite d'endpoints. Tout est en little-endian, **sauf la payload** quand la trame l'indique (cf. §4.2).

| offset | champ  | type   | description                                    |
|--------|--------|--------|------------------------------------------------|
| 0      | type   | u32 LE | = 0                                            |
| 4      | handle | 4 o    | device émetteur (même handle que ses annonces) |
| 8      | rtype  | u32 LE | type d'enregistrement — cf. §4.1               |

|    |         |          |                                                       |
|----|---------|----------|-------------------------------------------------------|
|    |         |          | △ constant par record : ce n'est PAS un compteur      |
| 12 | flags1  | u16+u16  | (a, b) : b = 30 (instrument) / 100 (radar) / 0 (0x2a) |
| 16 | port    | u16 LE   | port du record (cf. ci-dessous)                       |
| 18 | length  | u16 LE   | longueur de la payload                                |
| 20 | payload | length o | 1 ou 2 endpoints (§4.1)                               |

Un **endpoint** = 8 octets, dans l'ordre d'octets de la payload :

|    |       |     |                                             |
|----|-------|-----|---------------------------------------------|
| +0 | ip    | 4 o | adresse IPv4                                |
| +4 | port  | u16 | numéro de port                              |
| +6 | extra | u16 | 0 chez les nœuds – variable chez les radars |

Le champ **@16** est un **numéro de port** : il vaut le port de l'**endpoint unicast** de la payload sur tous les records de nœud. Chez les radars (0x28 / 0x29) et sur 0x2a, cette égalité tombe.

**rtype** n'est pas une clé unique (0x08 apparaît avec plusieurs ports) : c'est le couple (**rtype**, **port**) qui identifie un enregistrement. **flags1.a** est lié au **rtype**.

#### 4.1 Formes de payload par longueur ●

La **longueur** suffit à décoder la payload — le **rtype** n'intervient pas :

| length | structure                                                  | rtype observés       |
|--------|------------------------------------------------------------|----------------------|
| 8 o    | [IP:PORT] — un seul endpoint, unicast RayNet               | 07 0f 13             |
| 16 o   | [IP:PORT][IP:PORT] — groupe multicast, puis unicast RayNet | 08 09 1e 23 28 29 2a |
| 20 o   | [12 o ● non résolu][IP:PORT]                               | 10                   |

- Dans les records à **deux** endpoints, le **1<sup>er</sup>** est un **groupe multicast** (226.192.x.x pour les nœuds, 232.x.x.x pour le radar) et le **2<sup>e</sup>** l'**adresse unicast RayNet** du device → chaque record décrit un canal : « *je diffuse sur ce groupe, depuis cette adresse* ».
- Dans les records à **un** endpoint, il n'y a que l'adresse unicast RayNet.
- Le **extra** (u16 après le port) est **nul chez les nœuds** ; il ne varie que sur les records radar 0x28 / 0x29 (§4.3).
- Les 12 premiers octets du record 0x10 restent ● non résolus ; ils contiennent deux numéros de port de la plage du device, sans IP associée.

**Allocation régulière** ● : chaque device dispose d'une plage unicast contiguë 2050+n et d'une tranche multicast 226.192.<sel>.<base+i>:2560+i, où le **dernier octet du groupe et le port avancent ensemble** (sel = octet de sélection propre au device).

#### 4.2 Ordre des octets de la payload ●

L'en-tête est toujours en little-endian, mais la payload peut être big-endian. Le critère robuste, utilisé par le dissecteur : les mots d'adresse portent un marqueur de 2 octets (§5) dont la **position** tranche — marqueur en **fin** de mot ⇒ payload little-endian, en **tête** ⇒ big-endian. Repli : @16 == 0 ⇒ big-endian (seul le record 0x2a est BE, et son @16 est nul).

⚠ En **big-endian**, le **port aussi se lit en big-endian** — c'est un u16, pas un u32. La structure [IP][u16 port][u16 extra] est la seule qui reste cohérente dans les deux ordres d'octets.

Un seul type de record big-endian a été observé (0x2a) et tous les bits de son @16 sont nuls : le bit exact d'endianness n'est pas isolable, la règle reste ●. ● Ses ports tombent dans la plage éphémère, ce qui suggère un **service à port alloué dynamiquement** — cohérent avec un @16 nul (pas de port fixe à publier).

#### 4.3 Records radar : le champ **extra** ●

Sur les records radar (0x28 / 0x29), les IP et les ports ne bougent pas : **extra est le seul champ variable**. Sur l'un des records, il prend un grand nombre de valeurs distinctes évoquant une **énumération angulaire** (1 024 valeurs ≈ 1 024 spokes par tour, résolution classique chez Raymarine) ; sur l'autre, une valeur quasi constante. ● Compteur de tour / azimuth de spoke ou nonce : non tranché.

## 5. Adressage : unicast RayNet + groupes multicast (●)

Deux domaines d'adressage se côtoient dans les payloads, distingués par un marqueur de 2 octets (dont la **position** donne aussi l'ordre des octets, §4.2) :

| Domaine | Marqueur             | Forme       | Rôle                                                                              |
|---------|----------------------|-------------|-----------------------------------------------------------------------------------|
| A       | 0xc612 ( ... 12 c6 ) | 198.18.x.x  | ● <b>adresse unicast</b> du device sur le backbone SeaTalk-HS/RayNet              |
| B       | 0xe2c0 ( ... c0 e2 ) | 226.192.x.x | ● <b>groupes multicast</b> de diffusion des nœuds (octet 2 = sélecteur de device) |
| B'      | —                    | 232.x.x.x   | ● <b>groupes multicast SSM</b> du radar                                           |

Ce n'est **pas** un équipement « multi-homé sur deux interfaces » : le domaine A est l'**adresse** du device, le domaine B les **groupes** sur lesquels il diffuse. Chaque record de type 0 associe les deux (§4.1).

- 198.18.0.0/21 (RFC 2544) est une plage **détournée par Raymarine** comme backbone ; elle est traduite vers le 192.168.x.0/24 exposé en Wi-Fi. 226.192.0.0/16 et 232.0.0.0/8 sont des plages multicast (la seconde est le bloc **SSM** normalisé).
- Les radars Quantum sont un cas particulier : ils exposent **leur propre IP Wi-Fi** (le radar crée son propre point d'accès) tout en diffusant sur le backbone RayNet.

**Interprétation d'ensemble** : ray5800 est un protocole d'**annonce**, à deux étages — le type 1/2 annonce l'**équipement** (handle, modèle, nom, IP), le type 0 annonce ses **canaux de service** (groupe multicast:port + source unicast:port, un record par port). Aucune donnée de mesure n'y transite : les flux circulent sur les endpoints annoncés.

## 6. Comportement temporel (●)

- Beacon ~**0,72 s** par device, tous les équipements à la même cadence.
- **Ensemble d'équipements stable** : aucune apparition/disparition observée sur plusieurs heures de capture.
- rtype (offset 8) et descriptor d'annonce (offset 12) : **constants** par enregistrement/device — ce ne sont pas des compteurs.
- **Les endpoints annoncés (IP et ports) ne varient jamais** : c'est une table de canaux statique, réémission périodiquement.
- Seul champ à variation rapide : le **extra** des endpoints d'un des records radar (§4.3).

## 7. Questions ouvertes

Grammaire acquise (●) : cadrage réseau, les 3 types de message, la grammaire d'annonce (handle/IP/nom), la grammaire du type 0 (endpoints IP:PORT, allocation régulière des ports et des groupes), et le recoupement du port radar entre l'extension du type 2 et le record de type 0.

Restent à lever, idéalement avec vérité terrain (●/●) :

- **Quel service** écoute derrière chaque port annoncé.
- Nature exacte de **flags1** (b = 30/100 : cadence ? classe ?) et le critère exact d'endianness.
- Signification du **extra** variable des records radar (azimut de spoke vs nonce).
- Le champ **u1** (@4) et le **flag** (@54) des annonces, les 12 premiers octets du record 0x10, et le record big-endian 0x2a à ports éphémères.

# Protocole RayDB — TCP 23333 (Raymarine)

Outils associés dans ce dépôt :

- `dissectors/raymarine_raydb.lua` — dissecteur Wireshark
- `raydb_decode.py` — décodeur hors-ligne (pipe tshark)
- `raydb_client.py` — client + TUI (découverte mDNS, repli mcast 5800 → connexion 23333)

## 1. Vue d'ensemble

**RayDB** est un bus **publish/subscribe clé→valeur** exposé par le MFD Raymarine. Un client s'y connecte, s'annonce, s'abonne à des arborescences de chemins ( `data/#` , `diag/mfd/#` , ...), et le MFD **pousse** les valeurs correspondantes (position, cap, vent, profondeur, vitesses, tensions, versions firmware, etc.), d'abord l'état courant (« retained ») puis les mises à jour au fil de l'eau.

| Propriété          | Valeur                                                                                                                                                                                                     |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transport          | TCP, port 23333                                                                                                                                                                                            |
| Chiffrement        | <b>aucun</b> — <b>trafic en clair</b> , joignable directement (PAS tunnelé dans SSH)                                                                                                                       |
| Adresse du service | IP WiFi/LAN du MFD (celle qui émet les annonces de découverte)                                                                                                                                             |
| Endianness         | <b>little-endian</b> partout                                                                                                                                                                               |
| Découverte         | <b>mDNS/Bonjour</b> en premier ( <code>_raydb._tcp.local</code> , UDP 5353) ; <b>repli</b> sur le multicast <b>224.0.0.1:5800</b> (voir <code>dissectors/raymarine_5800.lua</code> ) si mDNS ne répond pas |
| Cadre applicatif   | plusieurs trames par segment TCP ; une trame peut être scindée sur plusieurs segments → <b>réassemblage requis</b>                                                                                         |

**mDNS** — l'enregistrement `_raydb._tcp` porte directement l'IP joignable du MFD ; seul son adresse est retenue, **le port annoncé (50000) est ignoré** : le bus RayDB décrit ici est sur 23333.

**5800 (repli)** — l'IP contenue *dans* l'annonce est une adresse interne `198.18.x.x` (réseau RayNet), **non joignable** depuis le WiFi. Toujours utiliser l'IP **source** du datagramme.

## 2. Cycle de vie d'une connexion

1. **Découverte** (optionnelle) : d'abord une requête **mDNS** sur `_raydb._tcp.local` — si un service répond, on prend l'IP de son enregistrement et on s'arrête là. À défaut (pas de réponse, ou pas de pile mDNS disponible), **repli** sur l'écoute des annonces multicast UDP 5800 : on retient l'IP source d'une annonce de MFD.
2. **Connexion TCP** vers `IP_MFD:23333` .
3. **HELLO** (op 7) : le client s'annonce ( `RayDBRemoteClient` ).
4. **SUBSCRIBE** (op 3) : un ou plusieurs abonnements ( `data/#` , ...).
5. **UPDATE** (op 4) : le MFD envoie l'état courant de tous les chemins couverts, puis les changements en continu.
6. **KEEPALIVE** : toutes les 5 s, le client envoie op 5 (compteur croissant), le MFD répond op 6 avec le même compteur.

### Diagramme de séquence

```
sequenceDiagram
    autonumber
    participant C as Client (RayConnect)
    participant M as MFD (RayDB :23333)

    Note over C,M: Découverte (hors bande) — mDNS d'abord
    C->>M: Requête mDNS _raydb._tcp.local (224.0.0.251:5353)
    M-->>C: Réponse SRV/A — IP du MFD (port annoncé ignoré)

    Note over C,M: Repli si mDNS reste muet
    M-->>C: Annonce (224.0.0.1:5800) — IP source = IP du MFD
```

```

Note over C,M: Établissement
C->>M: TCP SYN → 23333
M->>C: SYN/ACK, ACK

C->>M: HELLO op7 "RayDBRemoteClient"
C->>M: SUBSCRIBE op3 "data/#"
C->>M: SUBSCRIBE op3 "Settings/Data/-/7/13/-/-/-/-"

Note over M,C: État courant (retained), puis flux continu
M->>C: UPDATE op4 data/cog = <float32, radians>
M->>C: UPDATE op4 data/depth = <double, mètres>
M->>C: UPDATE op4 data/position = "<lat>,<lon>"
M->>C: UPDATE op4 ... (~90 chemins)

loop toutes les ~5 s
  C->>M: KEEPALIVE op5 (compteur = n)
  M->>C: KEEPALIVE-ACK op6 (compteur = n)
end

Note over M,C: Mises à jour poussées quand une valeur change
M->>C: UPDATE op4 data/sog = <float32>
M->>C: UPDATE op4 data/heading/true = <float32, radians>

```

### 3. Format d'une trame

Toutes les trames partagent le même en-tête. `len` couvre **tout ce qui suit le champ `len`** (il ne s'inclut pas lui-même).

| offset | taille | champ       | description                                        |
|--------|--------|-------------|----------------------------------------------------|
| 0      | u32    | len         | longueur du reste de la trame                      |
| 4      | u32    | msg_type    | toujours 1                                         |
| 8      | u8     | op          | opcode (voir §4)                                   |
| 9      | u32    | path_len    | longueur du chemin en octets                       |
| 13     | u32    | pad         | réservé / flags (0 observé)                        |
| 17     | ...    | path        | chemin ASCII (path_len octets, non terminé par \0) |
| 17+pl  | ...    | value_block | bloc valeur / trailer (voir §5), selon op          |

- **Multi-trames** : un segment TCP peut concaténer plusieurs trames.
- **Réassemblage** : lire `len`, attendre `4 + len` octets avant de décoder.

### 4. Opcodes (op)

| op | Nom           | Sens | Rôle                                                |
|----|---------------|------|-----------------------------------------------------|
| 3  | SUBSCRIBE     | C→S  | s'abonner à un chemin / sous-arbre ( /# )           |
| 4  | UPDATE        | S→C  | valeur d'un chemin (état initial puis changements)  |
| 5  | KEEPALIVE     | C→S  | ping périodique (~5 s), porte un compteur croissant |
| 6  | KEEPALIVE-ACK | S→C  | pong, ré-écho du compteur reçu                      |
| 7  | HELLO         | C→S  | annonce du client ( RayDBRemoteClient )             |

Les opcodes 0 / 100 / 115 aperçus avec un découpage naïf n'existent pas : ce sont des artefacts d'un désalignement dû à l'absence de réassemblage TCP. Avec réassemblage, seuls 3/4/5/6/7 apparaissent.

## 5. Bloc valeur

### 5.1 UPDATE (op 4)

```
[3 octets réservés][u32 type][payload...]
```

Les 3 octets réservés valent `00 00 00` sur les UPDATE, et le bloc se termine par 4 octets nuls. Le `type` détermine le format du payload ; l'énumération se lit comme une suite d'entiers de largeur croissante, suivie des composites :

| type | format du payload       | notes                                                               |
|------|-------------------------|---------------------------------------------------------------------|
| 0x00 | bool (1 octet)          | ex. <code>modeLedCtrl</code>                                        |
| 0x01 | i8 (1 octet)            | rare, vu en élément de liste                                        |
| 0x02 | i16 (2 octets)          | rare, vu en entrée de table                                         |
| 0x03 | i32 (4 octets)          | entier le plus courant ( <code>Diagnostics/CrashLogCount_*</code> ) |
| 0x04 | i64 (8 octets)          | rare ( <code>Settings/Data/-/10/2/...</code> )                      |
| 0x07 | u32 (4 octets)          | entier (ex. adresse CAN)                                            |
| 0x09 | float32 (4 octets)      | grandeurs analogiques (cap, vent...)                                |
| 0x0a | double (8 octets)       | profondeur, min/max...                                              |
| 0x0b | [u64 len][octets]       | chaîne (position, IP...)                                            |
| 0x0d | liste (voir ci-dessous) | <code>availableSysCarto</code> , <code>Settings/Data/...</code>     |
| 0x0e | table (voir ci-dessous) | <code>diag/versions/réseau</code> , <code>sf/...</code> , alertes   |

0x05 , 0x06 , 0x08 et 0x0c n'apparaissent dans aucune capture ; la suite laisse deviner `u8` , `u16` et `u64` , mais rien ne le vérifie.

**Types 0x0d (liste) et 0x0e (table)** — deux conteneurs comptés, qui ne diffèrent que par la présence d'une clé devant chaque élément :

```
0x0d [u64 n] puis n × ( [u32 type][payload] )
0x0e [u64 n] puis n × ( [u64 klen][clé (klen octets)][u32 type][payload] )
```

Le `payload` suit **immédiatement** son type et se lit selon le même barème que ci-dessus : il n'y a pas de champ de longueur générique. Seules les chaînes en portent un, et c'est le `[u64 len]` du type `0x0b` lui-même — d'où l'apparence trompeuse d'un `vlen` sur les exemples en chaîne.

⚠ Le `0x0e` n'est **pas** une « valeur nommée » : c'est une table de `n` entrées, et `n` vaut souvent plus de 1 (jusqu'à plusieurs dizaines). Un décodeur qui lit la première entrée et s'arrête perd le reste en silence : une entrée `data/exportedCamera/...` porte plusieurs champs (URI RTSP, IP, modèle...), une entrée d'`alertHistory/Records/...` de même (horodatage, titre, type...).

La clé réencodée double **souvent**, mais pas toujours, le dernier segment du chemin. D'où le préfixage conditionnel « nom = valeur » de `raydb_decode.decode_value` .

Les longueurs sont des `u64` mais restent petites ; un décodeur peut lire les 4 octets bas. (Piège Wireshark 4.6 : `TvbRange::le_uint()` ne gère que  $\leq 4$  octets.)

**Unités** — angles en radians, vitesses en m/s, profondeurs en mètres ; `data/position` est une chaîne `"latitude,longitude"` .

### 5.2 Requêtes HELLO / SUBSCRIBE (op 7 / 3)

Le bloc valeur est un « trailer » de 15 octets :

```
[3 octets réservés][u32 type = 0x07][u64 = 0]
```

Les 3 octets réservés sont **opaques** et **différent selon l'opcode** : `00 01 01` pour HELLO, `01 00 00` pour SUBSCRIBE. Ils sont identiques sur E70363 et E70481 → les reproduire tels quels garantit l'acceptation.

### 5.3 KEEPALIVE (op 5 / 6)

Trame de 32 octets, `path_len = 0`. Bloc valeur `[00 01 01][u32 0x07][u32 = 0][u32 compteur]` où le compteur s'incrémente à chaque ping ; le MFD renvoie op 6 avec le même compteur. Cadence observée : **1 ping / 5 s**.

Le compteur occupe donc les **4 derniers** octets — ceux qui servent de padding sur les UPDATE, où ils sont toujours nuls.

## 7. Espace de noms des chemins

- **data/...** — données de navigation live, ex. : `data/position`, `data/sog`, `data/cog`, `data/depth[/min|max/offset]`, `data/heading/{true,magnetic}`, `data/wind/{speed,direction}/{apparent,true}`, `data/pitch`, `data/roll`, `data/yaw`, `data/rot`, `data/rudder`, `data/bearing/variation`.
- **diag/mfd/<modèle> <série>/...** — diagnostics : `network/{ip_address, mac_address, sub_net_mask, link_status}`, `cartography_info/*_base_map_version`, `nmea2000_info/can_address`, `port*_status/*`, versions firmware.
- **Settings/Data/...** — réglages (ex. nom du bateau via `Settings/Data/-/7/13/-/-/-/-`).

Un abonnement se termine par `/#` pour couvrir tout un sous-arbre. L'espace de noms va bien au-delà de la télémétrie de navigation : configuration d'écran et pilotage de périphériques partagent le même arbre `data/`.

## 8. Correspondance RayDB → NMEA 0183

Table implémentée par `raydb_client.py --nmea`. Chaque phrase est déclenchée par **un seul** chemin, pour éviter les doublons ; les autres chemins ne font qu'alimenter un cache lu au moment de l'émission. La colonne « YDWG » indique si une passerelle Yacht Devices branchée sur le même bus émet aussi cette phrase (relevé dans `pcap/nmea_ydwg.pcap`).

| Phrase  | Chemin déclencheur                    | Chemins en cache                                                                                             | YDWG |
|---------|---------------------------------------|--------------------------------------------------------------------------------------------------------------|------|
| RMC     | <code>data/position</code>            | <code>data/sog</code> , <code>data/cog</code> , <code>data/bearing/variation</code>                          | ✓    |
| GGA     | <code>data/position</code>            | <code>data/position/altitude</code>                                                                          | ✓    |
| GLL     | <code>data/position</code>            | —                                                                                                            | ✓    |
| VTG     | <code>data/sog</code>                 | <code>data/cog</code> , <code>data/bearing/variation</code>                                                  | ✓    |
| HDT     | <code>data/heading/true</code>        | —                                                                                                            | ✓    |
| HDM     | <code>data/heading/magnetic</code>    | —                                                                                                            | ✓    |
| HDG     | <code>data/heading/magnetic</code>    | <code>data/bearing/variation</code>                                                                          | ✓    |
| GST     | <code>data/position/accuracy</code>   | —                                                                                                            | ✓    |
| MWV (R) | <code>data/wind/speed/apparent</code> | <code>data/wind/direction/apparent</code>                                                                    | ✓    |
| VWR     | <code>data/wind/speed/apparent</code> | <code>data/wind/direction/apparent</code>                                                                    | ✓    |
| MWV (T) | <code>data/wind/speed/true</code>     | <code>data/wind/direction/true</code>                                                                        | ✓    |
| VWT     | <code>data/wind/speed/true</code>     | <code>data/wind/direction/true</code>                                                                        | ✓    |
| MWD     | <code>data/wind/speed/true</code>     | <code>data/wind/direction/true</code> , <code>data/heading/true</code> , <code>data/bearing/variation</code> | ✓    |
| DPT     | <code>data/depth</code>               | <code>data/depth/offset</code>                                                                               | ✓    |
| DBT     | <code>data/depth</code>               | —                                                                                                            | ✓    |
| DBS     | <code>data/depth</code>               | <code>data/depth/offset</code>                                                                               | ✓    |

| Phrase | Chemin déclencheur | Chemins en cache                          | YDWG |
|--------|--------------------|-------------------------------------------|------|
| VHW    | data/stw           | data/heading/true , data/heading/magnetic | ✓    |
| ROT    | data/rot           | —                                         | ✓    |
| RSA    | data/rudder        | —                                         | ✓    |
| XDR    | data/roll          | data/pitch                                | ✓    |
| VDR    | data/tide/drift    | data/tide/set                             | ✓    |

**Référentiels** — `data/wind/direction/{true,apparent}` sont des angles **relatifs à l'étrave**, pas des directions référencées au nord. `MWD`, qui se réfère au nord, ajoute donc le cap. La formule se recoupe avec une passerelle YDWG branchée sur le même bus, qui émet `MWD`, `MWV` (T) et `HDT` : `MWD` y vaut bien `angle MWV_T + HDT`.

**Couverture** — 21 des 29 chemins `data/` sont exploités. Les 8 inutilisés sont des statistiques ou des grandeurs sans phrase NMEA évidente : `data/cog/stable`, `data/depth/{min,max}`, `data/sog/{avg,max}`, `data/stw/{avg,max}`, `data/yaw`.

**Écart avec la passerelle YDWG** — les 20 types produits par `raydb_client.py --nmea` sont tous émis aussi par le YDWG, qui en produit 36 au total. Les 16 types non couverts demandent des données absentes de `data/#` :

| Manque                                 | Phrases                                       | Source                                        |
|----------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Produisibles sans RayDB                | ZDA , DTM                                     | horloge système, datum fixe                   |
| Données absentes de <code>data/</code> | MTW , VLW , MDA , GSA , GSV , GRS             | température eau, loch, météo, satellites      |
| Navigation / route                     | APB , BOD , BWC , RMB , RTE , WPL , XTE , HTD | waypoints et pilote, hors <code>data/#</code> |

La navigation demanderait de souscrire à un autre espace de noms que `data/#` (l'arbre `Settings/...`, non exploré ici).

## 9. Notes d'implémentation

- **Réassemblage TCP obligatoire.** Bufferiser, lire `len`, ne décoder que lorsque `4 + len` octets sont disponibles ; boucler pour les trames multiples.
- **Piège tshark : `data.data` n'est pas réassemblé.** Avec `--only-protocols ...,data`, le dissecteur `data` ne demande jamais de déssegmentation : `data.data` vaut exactement la charge utile d'un segment (`len(data.data) == tcp.len`). Il remet les segments dans l'ordre et écarte les retransmissions — ce que `tcp.payload` ne fait pas — mais le recollage reste à la charge du décodeur, par flux **et par sens**.
- **Captures à trous.** Un segment manquant décale son flux définitivement. Le décodeur hors-ligne s'arrête alors sur ce flux plutôt que de deviner un recadrage ; Wireshark se resynchronise et rend quelques trames de plus, au prix d'une trame aberrante à la reprise.
- **Handshake reproductible.** Envoyer HELLO puis SUBSCRIBE `data/#` avec les octets exacts (cf. `build_hello` / `build_subscribe` dans `raydb_client.py`).
- **Keepalive.** Répondre/émettre le ping op5 toutes les 5 s pour éviter que le MFD ne considère la session comme morte (à confirmer selon les MFD ; les captures montrent le client comme initiateur du ping).
- **Robustesse du décodage.** Traiter les longueurs comme bornées, ignorer les blocs valeur trop courts, se rabattre sur un affichage hexadécimal si le `type` est inconnu.

Références : `dissectors/raymarine_raydb.lua`, `raydb_decode.py`, `raydb_client.py`, 1. `protocole-udp5800` (découverte 5800), 4. `ssh-mfd-analyse` (auth MFD).

# Protocole RRCE — télécommande Raymarine (TCP 50000)

Rétro-ingénierie du canal d'entrée utilisé par RayConnect pour piloter un MFD Raymarine à distance : touchers de l'écran, **boutons de façade** et **molette**.

## Vue d'ensemble

RayConnect affiche l'écran du MFD via un flux vidéo **RTSP (port 8554)** et renvoie les entrées de l'utilisateur sur un canal séparé : **TCP 50000**. Le MFD écoute sur ce port ; le client (téléphone/tablette, ou nos scripts) s'y connecte et pousse les événements.

- **Sens du trafic : strictement unidirectionnel** — client → MFD. Côté MFD, **aucun segment de données** en retour (que des ACK TCP vides) : pas de réponse applicative, pas de bannière.
- **Une seule connexion TCP persistante** par session. Le client ouvre depuis un port éphémère vers `MFD:50000` et l'emploie pour tous les événements jusqu'à la fin.
- **Aucun handshake applicatif** : pas de trame d'ouverture spécifique, on envoie directement des records dès la connexion établie.

La magie sur le fil est littéralement `ECRR ( 45 43 52 52 )` — mais **RRCE est bien le nom donné par Raymarine**, pas une invention de notre outillage (**code**) : la constante source est `"RRCE".getBytes()`, écrite à l'envers dans la trame (`f91a[3], f91a[2], f91a[1], f91a[0]`), autrement dit un u32 little-endian. Le nom se retrouve d'ailleurs dans le service mDNS `_rym_rrc._tcp` et dans la clé TXT `raymarine-mfd-rrc-version`.

## Découverte : d'où vient le port 50000

Le client ne code **jamais 50000 en dur (code)**. Il résout le service mDNS `_rym_rrc._tcp.local.`, dont l'enregistrement SRV donne l'hôte **et le port** ; le TXT fournit au passage `raymarine-mfd-rrc-version` (version du protocole, reportée dans l'en-tête, voir plus bas) et `raymarine-mfd-model`. La connexion n'est ouverte qu'après cette résolution, avec `TCP_NODELAY` et `SO_KEEPALIVE`. Un MFD annonçant un autre port serait donc suivi sans problème par l'app — nos outils, eux, ont 50000 par défaut ( `--port` pour en changer).

## Format de trame

Tous les records partagent un **en-tête de 9 octets**. L'octet [7] porte la **longueur de la charge utile** qui suit — c'est lui qui donne la taille du record, il ne faut donc pas supposer un pas fixe.

| offset | taille | champ        | valeur                                              |
|--------|--------|--------------|-----------------------------------------------------|
| 0      | 4      | magic        | "ECRR" = 45 43 52 52                                |
| 4      | 1      | version      | 01 — version de l'en-tête, jamais vue autrement     |
| 5      | 1      | rrc_ver      | version du protocole RRC (0a ou 00 selon le client) |
| 6      | 1      | type         | 01 = boutons · 02 = molette · 03 = écran tactile    |
| 7      | 1      | len          | longueur de la charge utile : 02 / 04 / 06          |
| 8      | 1      | réservé      | 00                                                  |
| 9      | len    | charge utile |                                                     |

Trois types de record, soit trois formats :

| type             | en-tête complet observé | charge utile | record           |
|------------------|-------------------------|--------------|------------------|
| 03 écran tactile | 01 0a 03 06 00          | 6 octets     | <b>15 octets</b> |
| 01 boutons       | 01 00 01 02 00          | 2 octets     | <b>11 octets</b> |
| 02 molette       | 01 00 02 04 00          | 4 octets     | <b>13 octets</b> |

**Le type tient dans le seul octet [6] (code)** : les trois constructeurs de trames de l'app ne diffèrent que par cet octet et la longueur. L'octet [5] qui le précède est une **variable statique commune aux trois**, initialisée depuis la clé TXT mDNS `raymarine-mfd-rrc-version` — c'est une version, pas un morceau de l'identifiant de source. C'est ce qui explique une bizarrerie du fil : dans `rrce.pcap`, **un même flux TCP** (port source 60284) porte `0a` sur ses records tactiles et `00` sur ses records boutons.

Conséquence pratique : **démultiplexer sur [6] seul**, et traiter [5] comme une version à afficher. Se caler sur [5..6] ferait passer pour inconnu un record parfaitement décodable émis par un MFD d'une autre version. Nos trois décodeurs ont été alignés là-dessus ; la règle ne fait qu'élargir la tolérance.

Un décodeur doit par ailleurs **se caler sur la longueur déclarée et non sur un pas fixe**, et remonter les types inconnus plutôt que les taire — c'est ainsi qu'ont été trouvés d'abord les boutons, puis la molette.

### Record tactile (15 octets)

| offset | taille | champ  | valeur                                       |
|--------|--------|--------|----------------------------------------------|
| 9      | 1      | op     | 1=DOWN 2=MOVE 3=UP 4=CANCEL                  |
| 10     | 1      | finger | identifiant de doigt (0, 1, ... multi-touch) |
| 11     | 2      | X      | u16 little-endian, normalisé 0..65535        |
| 13     | 2      | Y      | u16 little-endian, normalisé 0..65535        |

Deux précisions (**code**) sur ce que le client met dans ces champs :

- **finger** n'est pas un index de pointeur mais un **compteur** : incrémenté à chaque DOWN, remis à 0 au UP. D'où les valeurs 0 et 1 des captures — c'est le rang du doigt dans le geste en cours, pas son identité pour le système.
- **Sortir de l'écran fabrique un UP**. Si un MOVE tombe hors du cadre vidéo, le client n'émet pas ce MOVE : il envoie un **UP simulé**, coordonnées débordante ramenée sur le bord (0 ou 65535 selon le côté), et remet **finger** à 0. C'est l'explication la plus simple du **UP** orphelin sans **DOWN** de `rrce.pcap` : un geste terminé par une sortie de cadre.

### Record bouton (11 octets)

| offset | taille | champ | valeur                                   |
|--------|--------|-------|------------------------------------------|
| 9      | 1      | code  | bouton de façade (voir table ci-dessous) |
| 10     | 1      | état  | 1 = enfoncé · 2 = relâché                |

### Codes des boutons

Ce sont les **codes de touches virtuelles Windows** (`VK_*`), identiques aux `keyCode` JavaScript, réutilisés tels quels par Raymarine :

| code | VK Windows       | bouton                             |
|------|------------------|------------------------------------|
| 0x0d | VK_RETURN        | <b>OK</b>                          |
| 0x1b | VK_ESCAPE        | <b>Back</b>                        |
| 0x21 | VK_PRIOR (Page↑) | <b>zoom -</b>                      |
| 0x22 | VK_NEXT (Page↓)  | <b>zoom +</b>                      |
| 0x25 | VK_LEFT          | flèche <b>gauche</b>               |
| 0x26 | VK_UP            | flèche <b>haut</b>                 |
| 0x27 | VK_RIGHT         | flèche <b>droite</b>               |
| 0x28 | VK_DOWN          | flèche <b>bas</b>                  |
| 0x76 | VK_F7            | <b>Home</b>                        |
| 0x77 | VK_F8            | <b>WPT</b> (waypoint)              |
| 0x78 | VK_F9            | <b>Menu</b>                        |
| 0x7a | VK_F11           | <b>switch</b> (bascule de panneau) |

Cette table, déduite des captures, est **confirmée nom par nom (code)** : le client associe les codes à des identifiants de ressource explicites, non obfusqués — `home_button` → 118 (0x76), `wpt_button` → 119, `menu_button` → 120, `switch_button` → 122, `back_button` → 27 (0x1b), `range_out_button` → 33 (0x21, dézoom) et `range_in_button` → 34 (0x22, zoom). Aucune ambiguïté ne subsiste, en particulier sur le sens des deux codes de zoom. 0x0d (**OK**) ne vient pas d'un bouton mais de l'**appui au centre du joystick** (voir plus bas).

À noter : les boutons **et** la molette ne sont émis que si l'écran du terminal est jugé « grand » (tablette) ; sur téléphone, seul le tactile part.

### Répétition tant que le bouton est tenu

L'état 01 (enfoncé) n'est **pas envoyé une seule fois** : le client le **réémet toutes les ~8,3 ms (≈ 120 Hz)** tant que le bouton reste appuyé, puis envoie un unique 02 au relâchement — un appui un peu tenu produit donc des dizaines de records. Tout consommateur doit dédupliquer sur le front montant.

Les **flèches font exception** : elles ne se répètent pas, et **plusieurs peuvent être enfoncées simultanément** (0x27 + 0x26 = diagonale haut-droite), relâchées ensemble. On observe aussi des **02 redondants** (jusqu'à 2 relâchements pour un appui de 0x27) : le décodage doit les tolérer sans tenir d'état.

### Les flèches viennent d'un joystick analogique

L'exception s'explique (**code**) : les quatre codes flèches ne sont pas produits par un pavé directionnel, mais par un **stick analogique virtuel** — un disque qu'on pousse au doigt. Le client cumule les déplacements, borne le vecteur, puis classe son **angle** par  $|\cos \theta| = |x| / \|(x, y)\|$  :

| condition sur  $|\cos \theta|$  | secteur | code(s) émis |  $|-|-|-|$  |  $> 0.987$  | à  $\pm 9,3^\circ$  de l'horizontale | 0x25 **gauche** ou 0x27 **droite** |  $< 0.156$  | à  $\pm 9,0^\circ$  de la verticale | 0x26 **haut** ou 0x28 **bas** |  $0.642 \dots 0.766$  |  $40,0^\circ \dots 50,1^\circ$  | **deux** codes, la diagonale | ailleurs | — | **rien** |

Trois conséquences pour qui interprète ce flux :

- **Environ 69 % des directions n'émettent rien.** Les bandes actives couvrent  $\sim 113^\circ$  sur  $360^\circ$  : entre  $9^\circ$  et  $40^\circ$  d'un axe, le doigt bouge et aucune trame ne part. Ce n'est pas une perte de capture, c'est le comportement voulu.
- **Deux seuils de rayon**, lus dans les ressources de l'app : **2 dp** de zone morte (en dessous, rien) et **8 dp** de saturation. Course utile de 6 dp, soit  $\sim 4$  mm — le stick est minuscule, ce qui explique qu'on atteigne vite les secteurs francs.
- **Une seule direction est verrouillée par geste** : tant que le stick n'est pas revenu au neutre, aucune nouvelle flèche n'est émise. Un changement de direction en cours de glissé provoque d'abord le(s) 02 du secteur précédent, puis le verrouillage du nouveau dans la foulée. C'est ce mécanisme qui produit les relâchements groupés des diagonales.

Le **centre du stick est le bouton OK** (0x0d). Un appui long envoie 0d 01 puis 0d 02 au relâchement ; un tap envoie les deux **séparés par une pause de 50 ms figée dans le code** — d'où les paires appui/relâchement d'OK batchées dans un même segment TCP relevées plus haut.

### Record molette (13 octets)

| offset | taille | champ | valeur                                                          |
|--------|--------|-------|-----------------------------------------------------------------|
| 9      | 2      | delta | i16 little-endian — incrément signé du cran                     |
| 11     | 2      | cumul | i16 little-endian — somme des delta depuis le début de la salve |

Le second champ est **exactement la somme cumulée** du premier, sans rupture à l'intérieur d'une salve. Le client entretient donc lui-même l'accumulateur ; un récepteur peut indifféremment intégrer les **delta** ou lire le **cumul**.

**Record d'ouverture** : chaque salve commence par **e8 ff 00 00**, soit **delta = -24**, **cumul = 0**. Il marque le début du geste (remise à zéro de l'accumulateur) et **ne doit pas être appliqué** : le record suivant repart de son propre **delta**, sans tenir compte du -24.

Amplitudes observées : **delta** va de **24 à 61**, jamais moins de 24 — un cran vaut donc  $\sim 24$  unités, les valeurs supérieures traduisant l'accélération quand on tourne vite. Cadence de 20 à 100 ms entre crans, et salves de 3 à 30 records séparées par des secondes d'inactivité.

### Ce que mesure vraiment **delta**

(**code**) L'organe est une **molette circulaire** qu'on tourne au doigt, et **delta** est un **angle en degrés**. Le client suit l'angle absolu du doigt autour du centre, et n'émet un record que lorsque l'écart avec le dernier cran **atteint  $24^\circ$**  — d'où le plancher de 24 constaté sur le fil, qui n'est donc pas une unité arbitraire mais un **seuil d'émission**. Deux détails confirmés :

- **cumul** est bien tenu par le client (**cumul += delta** à chaque cran), et remis à zéro à l'**onDown** : le record d'ouverture porte **cumul = 0** **par construction**, quel que soit son **delta**. C'est ce qui justifie de le reconnaître sur **cumul == 0** plutôt que sur **delta == -24**.

- Un garde-fou anti-passage-par-0° inverse le signe quand l'écart dépasse 270°, pour qu'un franchissement du zéro ne soit pas lu comme un tour complet à l'envers.

**Divergence de version** : le plancher de 24 est stable, mais pas le plafond — ne pas coder de borne supérieure.

### Batching

Plusieurs records peuvent être concaténés dans un même segment TCP — deux doigts qui bougent ensemble, ou l'appui **et** le relâchement d'un bouton OK dans le même segment ( `...0d 01 + ...0d 02` , tap instantané, observé 3 fois). Chaque record ré-inclut la magie + l'en-tête complet (pas d'agrégation d'en-tête). Le découpage suit la longueur déclarée, en resynchronisant sur `ECRR` si un segment TCP coupe un record en deux.

## Structure d'un geste

Un geste est encadré par un DOWN et un UP, avec des MOVE intermédiaires. L'appariement DOWN/UP est **la règle mais pas une garantie** : on observe des UP isolés sans DOWN correspondant (même symptôme que les relâchements redondants des flèches). Un décodeur ne doit donc pas supposer un appariement strict.

- **op 4 (CANCEL)** est géré par les outils mais **jamais observé** dans les captures.
- L'**en-tête tactile** `01 0a 03 06 00` est **identique** sur les deux familles de MFD (Axiom 7 et Axiom Pro) : le format est commun.
- **finger** vaut 0 en mono-touch, 0 et 1 en multi-touch (pincer-zoomer). Pas de valeur > 1 observée.

### Coordonnées

X et Y sont **normalisés sur 0..65535**, indépendants de la résolution réelle de l'écran (le client convertit lui-même pixels → fraction × 65535). Pour repasser en pourcentage : `pct = val / 655.35` .

## Séquences types

- **Tap** : `DOWN f0 (x,y)` → courte pause → `UP f0 (x,y)` .
- **Swipe** : `DOWN f0 (x1,y1)` → série de `MOVE f0 (...)` interpolés → `UP f0 (x2,y2)` .
- **Pinch/zoom** : mêmes ops mais deux `finger` (0 et 1) évoluant en parallèle, souvent batchés dans les mêmes segments.
- **Appui bouton** : `code 01` → répété à 120 Hz tant que tenu → `code 02` .
- **Salve molette** : record d'ouverture ( `cumul = 0` ) → suite de crans signés, le `cumul` suivant le sens de rotation, changements de sens compris.
- **Joystick** : un seul secteur verrouillé à la fois, une ou deux flèches enfoncées ensemble (diagonale), relâchées ensemble, sans répétition.

## Outillage du projet

- **rrce\_touch.py** — injecte des entrées dans un MFD réel ( `tap` , `swipe` , `key` , `wheel` , `raw` ) ou rejoue une capture ( `--replay` , tous types de records, y compris les sources non décodées, rejouées telles quelles). Coordonnées en entier 0..65535, en fraction `--frac` , ou en pixels `--screen LxH` . `--dry-run` affiche les octets sans rien envoyer, `--list-keys` liste les noms de boutons acceptés.
- **rrce\_sniff.py** — décode en direct (ou hors-ligne via `-r pcap/...` ) les trames RRCE et affiche DOWN/MOVE/UP, doigt, X/Y et pourcentage, ainsi que les appuis de boutons (avec marquage des répétitions ; `--keys` masque le tactile), les crans de molette et **les records de source inconnue**, affichés en hexa. Buffer par flux TCP + resync sur `ECRR` pour tolérer les segments coupés.
- **dissectors/raymarine\_rrce.lua** — dissecteur Wireshark ( `tcp.port == 50000` ), champs `rrce.type` / `rrce.rrc_version` , `rrce.op` / `rrce.x` / `rrce.y` , `rrce.key` / `rrce.key_state` , `rrce.wheel_delta` / `rrce.wheel_total` , avec déségmentation des records à cheval sur plusieurs segments et alerte « expert » si la magie `ECRR` manque (désynchronisation). Le champ `rrce.source` (u16 sur [5..6]) a été remplacé par le couple `rrce.rrc_version + rrce.type` : un filtre existant est à réécrire.
- **mfdsim** — le simulateur écoute sur 50000 et journalise les trois sortes de records ; un type inconnu ou un code de bouton inconnu sont affichés en hexa (niveau `warning` ) plutôt qu'ignorés (voir `mfdsim/mfdsim/rrce.py` ).

## Points de sécurité

Le canal est **non authentifié et non chiffré**. Quiconque a un accès réseau au MFD (le point d'accès Wi-Fi du bateau) peut ouvrir une connexion TCP 50000 et injecter touchers, appuis de boutons **et rotations de molette** arbitraires — soit un contrôle total de l'interface du MFD à distance, sans aucune validation côté appareil.

# Analyse du SSH du MFD Raymarine — synthèse exhaustive

---

## Ouvrir une session SFTP

Le credential d'accès `media_rw` est le champ `SshPrivateKey` du fichier `user_settings_<uuid>.json` de l'app **RayConnect**. `rm_ssh.py` l'exploite directement :

```
./rm_ssh.py path_to/user_settings_<uuid>.json      # ouvre une session SFTP sur  
media_rw@MFD
```

La clé publique correspondante n'est pas contenue dans le firmware : elle a été enrôlée sur ce MFD précis. Elle ne rouvrira donc que les MFD sur lesquels RayConnect s'est déjà appairé.

# Protocole de messages Raymarine — TCP 8182

Protocole de messages du MFD exposé sur le port **8182**. Plusieurs services y coexistent (accès SSH, cartographie, espace disque, propriété — cf. §6) ; cette doc détaille le service **SSHAccess** (enrôlement de la clé publique SSH par RayConnect), le seul observé en capture.

**Niveau de confiance** : ● confirmé par les données · ● hypothèse forte · ● non résolu.

Outils associés dans ce dépôt :

- `dissectors/raymarine_8182.lua` — dissecteur Wireshark ( `tcp.port == 8182` )

## 1. Ce que résout ce protocole

La doc `4. ssh-mfd-analyse` établissait que le MFD s'authentifie **par clé**, que la clé du compte `media_rw` n'était pas dans le firmware et donc laissait **ouverte** la question : \*par quel canal la clé publique atterrit-elle dans le MFD ?

**Réponse : le port TCP 8182.** RayConnect y pousse son identité et sa clé publique SSH ; le MFD (le `PlatformServicesDaemon`) l'enregistre et accuse réception. La clé enrôlée est **exactement** celle du `user_settings_*.json` (champ `SshPrivateKey`), celle qu'exploite `rm_ssh.py` pour se connecter en SFTP.

## 2. Cadre réseau (●)

| Élément              | Valeur                                                                        |
|----------------------|-------------------------------------------------------------------------------|
| Serveurs             | 3 MFD (voir ci-dessous), port <b>8182</b>                                     |
| Client               | le poste où tourne RayConnect, port éphémère                                  |
| Transport            | <b>TCP</b> , en clair                                                         |
| Échange              | 1 requête client → 1 réponse serveur, puis fermeture                          |
| Cadence              | ré-enrôlement à chaque connexion (≈ toutes les 7 s tant que l'app est active) |
| Handshake applicatif | <b>aucun</b> : la requête part dès la connexion établie                       |

Le protocole est observé **identique sur 3 MFD de 2 modèles** (AXIOM 7 et AXIOM PRO 9 S) — même cadrage à l'octet près (commande, `appType`).

Toutes les connexions, **tous MFD confondus**, portent le **même** message (commande `1500000`, `appType 2`) avec la **même** identité et la **même** clé — le cadrage ne dépend donc pas de l'appareil (cf. §8).

## 3. Format de trame (●)

Little-endian partout. Le premier champ est un **u32 de commande** (et non un opcode d'un octet suivi d'un tag). Layout **confirmé à l'octet près** par `Messages::sendSSHKey` dans `libwp.so` (la fonction native qui construit la trame).

La commande est un **MessageType propre au service**, et non un opcode global : `1500000` / `1500001` = Request/Response y sont **réutilisés par plusieurs services** (SSHAccess, Chart...), c'est le champ **appType qui sélectionne le service** (=2 = SSHAccess). « SSHAccessRequest » désigne donc (service SSHAccess) + (MessageType Request `1500000`) — voir la carte des commandes en §6.

| Sens    | Octets sur le fil        | Commande (u32 LE)       | Décimal   | Nom (libwp.so)                                 |
|---------|--------------------------|-------------------------|-----------|------------------------------------------------|
| Requête | <code>60 e3 16 00</code> | <code>0x0016E360</code> | 1 500 000 | <b>SSHAccessRequest</b> (MessageTypeRequest)   |
| Réponse | <code>61 e3 16 00</code> | <code>0x0016E361</code> | 1 500 001 | <b>SSHAccessResponse</b> (MessageTypeResponse) |

**Requête — SSHAccessRequest (commande 1 500 000)**

| offset | taille | champ   | description                                              |
|--------|--------|---------|----------------------------------------------------------|
| 0      | u32    | command | 0x0016E360 = 1500000 (identifie la requête)              |
| 4      | u32    | len     | longueur du reste de la trame (octets qui suivent)       |
| 8      | u32    | appType | 2 (nommé « appType » dans libwp.so)                      |
| 12     | u32    | msgType | 0 (SSHAccessResponseMessageType ; None en requête)       |
| 16     | u32    | id_len  | longueur de l'identité (email)                           |
| 20     | ...    | id      | identité ASCII (email de compte de RayConnect)           |
| 20+idl | ...    | pubkey  | clé publique OpenSSH ("ssh-rsa AAAA..."), jusqu'à la fin |

La clé est le **dernier champ** : pas de longueur propre, elle court jusqu'au bout de la trame (`len` la borne).

**Réponse — SSHAccessResponse (commande 1 500 001)**

|    |     |         |                                                      |
|----|-----|---------|------------------------------------------------------|
| 0  | u32 | command | 0x0016E361 = 1500001 (= requête + 1)                 |
| 4  | u32 | len     | longueur du reste (= 34 dans l'exemple)              |
| 8  | u32 | appType | 2 (ré-écho de l'appType)                             |
| 12 | u32 | msgType | 1 (SSHAccessResponseMessageType ; 1 = KeyAddSuccess) |
| 16 | u32 | id_len  | longueur de l'identité                               |
| 20 | ... | id      | identité ré-écho (email)                             |

La réponse **ne contient pas** la clé : elle ré-écho l'identité et porte le résultat dans `msgType` (SSHAccessResponseMessageType) :

| Valeur | Nom            | Sens                                                          |
|--------|----------------|---------------------------------------------------------------|
| 0      | None           | non renseigné (valeur en requête)                             |
| 1      | KeyAddSuccess  | clé ajoutée — <b>seule valeur observée</b> sur les 5 captures |
| 2      | KeyAddFail     | échec d'ajout de la clé                                       |
| 3      | AuthRejected   | autorisation refusée                                          |
| 4      | AuthInProgress | autorisation en cours                                         |

**5. La clé enrôlée = la clé de RayConnect (🟢)**

La `pubkey` envoyée sur 8182 est **identique octet pour octet** à la clé publique dérivée de la clé privée du `user_settings_*.json` :

```
ssh-keygen -y -f <clé du user_settings>
# ssh-rsa AAAAB3NzaC1yc2E...
```

Le cycle d'accès complet est donc :

1. RayConnect → 8182 : enrôle {identité, pubkey} ← ce protocole
2. MFD : écrit la pubkey dans l'`authorized_keys` de `media_rw`
3. RayConnect → 22 : SFTP en `media_rw` avec la clé privée correspondante

C'est ce que reproduit `rm_ssh.py` à l'étape 3 (il a la clé privée) ; l'étape 1 explique **comment** cette clé a été autorisée en premier lieu.

**Clé par installation, pas par MFD** : la même clé est enrôlée à l'identique sur tous les MFD (cf. §2). RayConnect génère donc **une** paire à l'installation (stockée dans le `user_settings_*.json`) et la ré-enrôle sur *chaque* MFD qu'il rencontre — un seul appareil client peut ainsi accéder à toute la flotte.

**6. Côté client : d'où vient la trame (🟢)**

La trame est bâtie en **code natif** (`libwp.so`), d'où l'absence de la commande `0x0016E360` dans le bytecode Java. La décompilation de `libwp.so` montre **deux** fonctions qui émettent sur ce protocole :

```
// clé SSH seule → command 1500000 (SSHAccessRequest) – CE QU'ON CAPTURE
Messages::sendSSHKey(socket, appType, email, sshPublicKey) {
    putInt(1500000); // command
    putInt(email+key+0xc); // len
    putInt(appType); // appType (=2)
    putInt(0); // msgType = 0 (None)
    putInt(email_len); ... email ... key
}

// clé SSH + certificat → command 1500007 (MessageTypeRequestOwnership)
Messages::sendRequestOwnerAuthCommand(socket, appType, username, sshKey, certKey) {
    putInt(1500007); putInt(len); putInt(appType); putInt(0);
    putInt(username_len); ... username ... sshKey ... certKey
}
```

`sendSSHKey` reproduit **exactement** la trame observée (§3) : c'est la preuve que le layout documenté est juste. `rm_ownerauth.py` émet l'autre, la `1500007`.

Le JNI `externalSendSshKey(host, 8182, username, publicKey, certKey)` (`BaseRaymarineTransferDriver`) appelle `sendRequestOwnerAuthCommand` — donc la variante **avec certificat** (`1500007`), dans le flux « owner request » (`RayDevicesManager.ownerRequest(userEmail, host, sshPublicKey, certPublicKey)`) : enrôler sa clé, c'est **revendiquer la propriété** du bateau, d'où l'email.

Correspondance des champs du fil (message `1500000` capturé) :

| Origine (libwp.so)            | Champ 8182                       | Valeur                                                            |
|-------------------------------|----------------------------------|-------------------------------------------------------------------|
| <code>email / username</code> | identité                         | l' <b>email</b> du compte ( <code>owner@example-boat.org</code> ) |
| <code>sshPublicKey</code>     | clé publique                     | la <b>clé SSH</b> du <code>user_settings_*.json</code>            |
| <code>certKey</code>          | (pas dans <code>1500000</code> ) | certificat — envoyé par <code>1500007</code> , cf. §8             |

**Divergence capture ↔ décompilation** : nos 5 captures montrent `1500000` (`sendSSHKey`, clé seule), alors que le JNI du binaire décompilé passe par `1500007` (clé + cert). `sendSSHKey` n'a d'ailleurs aucun appelant résolu dans le décompilé. Explication la plus probable : **versions d'app différentes** — l'app a migré de l'envoi de clé simple vers la revendication de propriété.

### Carte des commandes (énums `libwp.so`)

Le même port 8182 et le même cadrage portent plusieurs services, chacun avec sa famille de `MessageType` autour de `1500000` (d'où l'importance de `appType`) :

| Service (énum)                    | Commandes                                                                                                                                                                                 |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SSHAccessMessageType</code> | <code>1500000</code> Request · <code>1500001</code> Response                                                                                                                              |
| <code>MessageTypeChart</code>     | <code>1500000</code> Request · <code>1500001</code> Response · <code>1500002/1500003</code> UpdateData · <code>1500004</code> LoaderStatus · <b><code>1500007</code> RequestOwnership</b> |
| <code>MessageTypeDiskspace</code> | <code>1500005</code> Request · <code>1500006</code> Response                                                                                                                              |

Deux énums de statut accompagnent les réponses : `SSHAccessResponseMessageType` (§3, l'ajout de clé) et `ChartResponseUpdateDataMessageType` (progression de mise à jour de carte). L'enrôlement de clé n'est donc qu'**un service parmi d'autres** exposés sur 8182 (cartographie, espace disque, propriété...).

## 7. Sécurité : enrôlement en clair, accepté sans étape visible (🟢)

Le canal est **en clair**. La connexion s'ouvre et la clé part immédiatement, sans secret partagé ni preuve de possession de la clé privée ; l'identité (`email`) est **purement déclarative**.

**Conséquence : quiconque a un accès réseau au MFD (le Wi-Fi du bord) peut enrôler sa propre clé publique** en envoyant une `SSHAccessRequest` (commande `1500000`) sur le port 8182, puis se connecter en **SFTP `media_rw`** avec accès à `/data/media/0/UserData`. C'est une porte plus large que le SFTP lui-même : ce dernier exige déjà une clé autorisée ; 8182 permet précisément de s'en autoriser une.

Le mot de passe Wi-Fi du MFD reste le seul rempart. Une fois sur le réseau, la chaîne 8182 → 22 donne un accès fichiers persistant.

## 8. Questions — résolues et ouvertes

- **En-tête** `60 e3 16 00` : opcode + tag ? **Résolu** (●) : c'est un **u32 de commande** unique ( `0x0016E360` = 1 500 000 en requête, `+1` en réponse), confirmé au désassemblage de `libwp.so` (§3) — pas un octet d'opcode suivi d'un tag. Identique sur les 3 MFD.
- Le protocole vaut-il au-delà d'un MFD ? **Résolu** (●) : observé identique sur AXIOM 7 **et** AXIOM PRO 9 S — commun à la gamme.
- **Le certificat** ( `certKey` ) : où passe-t-il ? **Résolu** (●) : sous la commande **1500007** (**MessageTypeRequestOwnership**), via `sendRequestOwnerAuthCommand{username, sshKey, certKey}` (§6). Pas dans la trame `1500000` capturée, d'où son absence. **Non observé en capture** : c'est cette commande que rejoue `rm_ownerauth.py`.
- **Autorisation** : `msgType` définit `AuthRejected` (3) et `AuthInProgress` (4) — il **existe** donc un mécanisme d'approbation côté MFD. Mais toutes les réponses observées valent `KeyAddSuccess` (1), sans étape visible. Quand l'autorisation est-elle exigée (premier appairage ? un réglage ? mode propriétaire) ? ●
- **Rôle de l'identité** : l'email est-il stocké côté MFD (commentaire de la ligne `authorized_keys` ? clé de déduplication ?) ou seulement ré-écho ? ●
- **Persistance** : `/mnt/tmp/...` suggère un montage volatil — l'enrôlement survit-il à un redémarrage, d'où le ré-enrôlement périodique ? ●

Références : 4. `ssh-mfd-analyse` (auth par clé, fichiers `authorized_keys`), `rm_ssh.py` (connexion SFTP avec la clé du `user_settings.json`).